

T.C.
SİVAS CUMHURİYET ÜNİVERSİTESİ
Şarkışla Uygulamalı Bilimler Yüksekokulu

NESNELERİN İNTERNETİ 2
GRUP PROJE RAPORU

ESP32 Tabanlı Metanoia Robot Kol ve Konveyör Sisteminin
M2M Haberleşmesi ve Sürü Mimarisi Açısından Değerlendirilmesi

Grup Adı / No	Metanoia
Grup Üyesi 1	Emirhan ERTÜRK — 2023481038
Grup Üyesi 2	Yusuf Rasim ERTÜRK — 2024481261
Grup Üyesi 3	Savaş MESTER — 2023481006
Grup Üyesi 4	İbrahim Buğrahan KARATEPE — 2023481009
Ders / Kod	Nesnelerin İnterneti 2
Dönem	Bahar Dönemi
Danışman	Dr. Öğr. Üyesi Mehmet Akif BUZPINAR



ÖZET

Bu çalışmada, Endüstri 4.0 ve IoT standartlarına uygun olarak siber-fiziksel süreçleri simüle eden, Makineden Makineye (M2M) haberleşme tabanlı bir otomasyon hücresi (Metanoia) tasarlanmıştır. Temel amaç, birbirinden bağımsız çalışan ESP32 tabanlı düğümlerin (robot kol ve konveyör) sürü (lider-takipçi) mimarisiyle kablosuz ağ üzerinden senkronize edilmesidir. Yöntem olarak, MQTT protokolü kullanılmış ve bulut tabanlı bir broker (HiveMQ) üzerinden Node-RED ağ geçidine entegrasyon sağlanmıştır. Geliştirilen sistemde, Takipçi düğüm (konveyör) ultrasonik sensör ile nesneyi algıladığında otonom olarak Lider düğüme (robot kol) bulut üzerinden görev emri iletmektedir. Elde edilen temel bulgular, yazılımsal ölü bant (deadband) algoritmalarıyla filtrelenen analog telemetri verilerinin ağdaki paket yığılmalarını önlediğini ve dağıtık IoT cihazlarının düşük gecikmeyle stabil çalışabildiğini göstermiştir. Sonuç olarak, karmaşık kablo altyapılarına ihtiyaç duymadan, yüksek kararlılıkta M2M haberleşen bir sistem prototiplenmiştir.

Anahtar Kelimeler: Nesnelerin İnterneti, Sürü Robotiği, M2M Haberleşme, MQTT Protokolü, Edge Computing.

1. GİRİŞ

1.1 Çalışmanın Amacı ve Kapsamı

Bu çalışmanın temel amacı, endüstriyel taşıma ve konumlandırma (Pick and Place) süreçlerini kablosuz IoT ekosistemine taşıyarak otonom bir makine ağı oluşturmaktır. Çalışma, ESP32 tabanlı iki farklı düğümün (Lider Robot Kol ve Takipçi Konveyör) internet üzerinden haberleşmesi, HiveMQ broker entegrasyonu ve telemetri verilerinin Node-RED ile izlenmesi sınırları içerisinde gerçekleştirilmiştir.

1.2 Problemin Tanımı

Geleneksel otomasyon sistemlerindeki kablo karmaşası ve donanım bağımlılığı, akıllı fabrika konseptinde yerini kablosuz ağlara bırakmaktadır. ESP32 tabanlı robot sürülerinde bulut haberleşmesinin sağlanmasındaki en büyük teknik zorluklar; Wi-Fi ağ gecikmeleri (latency), analog sensör parazitlerinin ağda paket tıkanıklığı (DDOS etkisi) yaratması ve asenkron sistemlerin senkronizasyonudur.

1.3 Çalışmanın Önemi

Bu sistem, farklı görevlere sahip otonom makinelerin merkezi bir PLC olmadan, doğrudan internet üzerinden (M2M) işbirliği yapabileceğini kanıtlamaktadır. Geliştirilen jitter (titreme) önleyici yazılımsal ağ filtreleri, ücretsiz bulut platformlarının endüstriyel prototiplemede çökmeden kullanılabilmesini sağlayarak literatüre özgün bir pratik katkı sunmaktadır.

1.4 Raporun Yapısı

Rapor yedi ana bölümden oluşmaktadır. İkinci bölümde sürü robotiği ve MQTT altyapısı, üçüncü bölümde sistemin donanım/ağ mimarisi, dördüncü bölümde entegrasyon ve kurulum, beşinci

bölümde test bulguları ve tartışma, altıncı bölümde genel sonuçlar sunulmuş ve son bölümde kaynaklar listelenmiştir.

2. Proje Tanımı

2.1 Sürü Robotiği Yapısı

Sürü robotiği, doğadaki kolektif davranışlardan ilham alan ve birden fazla basit robotun karmaşık hedefleri ortaklaşa çözmesini sağlayan bir sistemdir. Metanoia projesinde "Lider-Takipçi" mimarisi benimsenmiştir. Konveyör sistemi (Takipçi), sahadaki çevresel veriyi (mesafe) toplayarak eylemi başlatır. Robot Kol (Lider) ise takipçiden gelen ağ sinyaliyle harekete geçerek fiziksel manipülasyonu (hedef görevi) tamamlar.

2.2 M2M Haberleşme ve Protokolü

Makineden Makineye (M2M) haberleşme, insan müdahalesi olmadan cihazların veri alışverişi yapmasıdır. Bu projede Publish/Subscribe (Yayınla/Abone Ol) modeliyle çalışan, düşük bant genişliği tüketen hafif MQTT protokolü kullanılmıştır. Cihazlar doğrudan birbirine bağlanmak yerine, merkezi bir Broker üzerinden belirli "Topic" (Konu) kanallarına abone olarak veri dinlemekte ve QoS (Hizmet Kalitesi) seviyesi 0 (At ve Unut) ile en düşük gecikmeyi hedeflemektedir.

2.3 ESP32 Mikrodenetleyici Platformu

ESP32, çift çekirdekli Xtensa LX6 işlemcisi, dahili Wi-Fi (802.11 b/g/n) ve Bluetooth yetenekleriyle uç bilişim (Edge Computing) için idealdir. Donanımsal PWM zamanlayıcıları motorları titreşimsiz sürerken, ağ katmanı eşzamanlı olarak arka planda TCP/IP yığını işleyerek IoT projelerinde yüksek kararlılık sağlar.

2.4 Bulut Platformları

Sistemde mesajlaşma omurgası olarak açık kaynaklı bulut broker olan HiveMQ kullanılmıştır. Görselleştirme ve SCADA ağı için ise Node-RED tercih edilmiştir. Node-RED, MQTT düğümleri üzerinden gelen ham verileri web tabanlı kontrol panellerine (Dashboard) dönüştürmektedir.

2.5 İlgili Çalışmalar Özet Tablosu

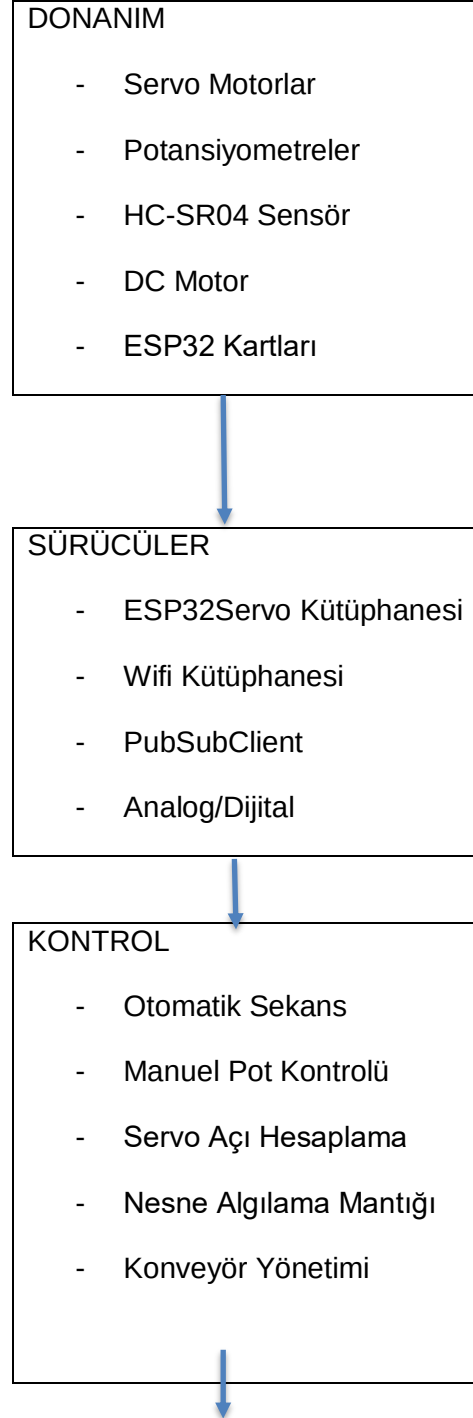
Kaynak	Konu	Yöntem	Çalışmaya Katkısı
[1]	MQTT Tabanlı IoT Sistemleri	Pub/Sub Mimarisi İncelemesi	Haberleşme protokolü seçimi.
[2]	ESP32 ile Uç Bilişim	Sensör verilerinin lokalde filtrelenmesi	Deadband filtresinin tasarımı.
[3]	Node-RED Endüstriyel Arayüzleri	Görsel Programlama ile Dashboard	SCADA kontrol paneli oluşturulması.

Tablo 2.1: İlgili çalışmalar özet tablosu

3. SİSTEM TASARIMI

3.1 Genel Sistem Mimarisi

Mimarî; Lider robot (Robot Kol), Takipçi robot (Konveyör) ve HiveMQ Bulut Broker katmanlarından oluşmaktadır. Takipçi sahayı izler, Broker'a veri yollar; Lider bu veriyi çeker ve uygular.



HABERLERLEŐME

- MQTT Broker
- Node – Red Dashboard
- metanoia/komut
- metanoia/durum
- metanoia/kontrol

Őekil 3.1: Genel sistem mimarisi

3.2 Robot Donanım Tasarımı

3.2.1 Ortak Bileőenler (Her Robot)

Bileően	Model / Deęer	Görev
Mikrodenetleyici	ESP32 DevKit V1	Uç biliőim ve Wi-Fi/MQTT haberleőmesi.
Güç Kaynaęı	5V ve 12V Adaptör	Sistem enerji izolasyonu ve beslemesi.

Tablo 3.1: Ortak donanım bileőenleri

3.2.2 Lider ve Takipçiye Özgü Bileőenler

Bileően	Model / Deęer	Görev
Motor Sürücü (Takipçi)	L298N	Konveyör DC motorunu sürmek.
Mesafe Sensörü (Takipçi)	HC-SR04	Nesne algılamak ve aę emrini tetiklemek.
DC Motor (Takipçi)	12V Redüktörlü	Bant hareketini saęlamak.
Servo Motor (Lider)	MG996R (4 Adet)	Robot kol manipölasyonunu saęlamak.

Tablo 3.2: Lider robota özgü bileőenler

3.3 Haberleşme Mimarisi

3.3.1 Yerel M2M Haberleşmesi (Robotlar Arası)

Robotlar arası lokal etkileşim tamamen MQTT üzerinden sağlanmaktadır. Sistemde standart port (1883) kullanılarak IoT iletişimi gerçekleştirilmiş, veri güvenliği için benzersiz Client ID'ler atanmıştır.

MQTT Topic	Yayımcı (Publisher)	Abone (Subscriber)	İçerik
metanoia/komut	Takipçi (Konveyör)	Lider (Robot Kol)	AL (Otonom başlatma komutu)
metanoia/durum	Lider (Robot Kol)	Takipçi (Konveyör)	BITTI (Süreci sonlandırma)

Tablo 3.3: Yerel M2M MQTT topic yapısı

3.3.2 Lider–Bulut Haberleşmesi

Node-RED üzerinden Lider robotun kontrolü ve telemetri izlemesi, belirli periyotlarla filtre edilmiş verilerin buluta basılması (Telemetry) mantığıyla çalışmaktadır.

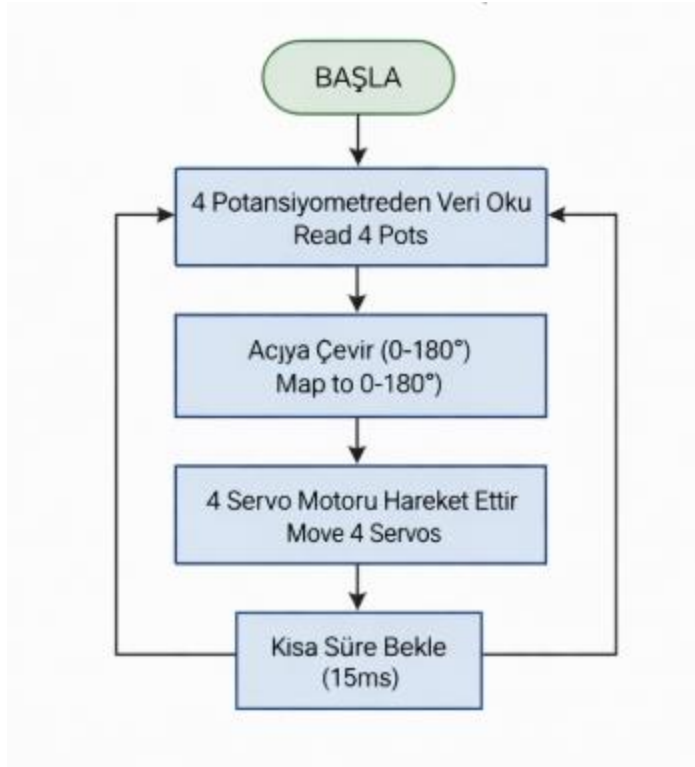
Bulut Topic	Yön	İçerik
metanoia/kontrol/govde	Bulut -> Lider	Node-RED'den gelen Gövde referans açısı
metanoia/kontrol/omuz	Bulut -> Lider	Node-RED'den gelen Omuz referans açısı
metanoia/kontrol/dirsek	Bulut -> Lider	Node-RED'den gelen Dirsek referans açısı
metanoia/kontrol/kiskac	Bulut -> Lider	Node-RED'den gelen Kıskaç referans açısı
metanoia/sistem_log	Lider -> Bulut	Sistemin güncel durum logları

Tablo 3.4: Lider–Bulut MQTT topic yapısı

3.4 Yazılım Mimarisi

3.4.1 Lider Robot Yazılım Akışı

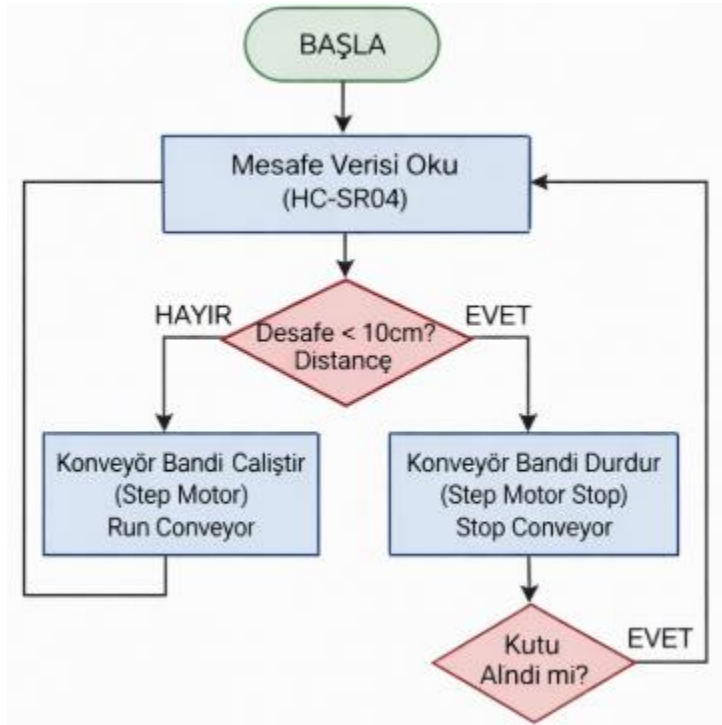
Lider robot setup() içinde Wi-Fi ve MQTT broker'a bağlanır, servo pinlerini PWM için hazırlar. loop() döngüsünde ise client.loop() ile bulutu dinler, asenkron callback fonksiyonuyla gelen komutları işler.



Şekil 3.2: Lider robot yazılım akışı

3.4.2 Takipçi Robot Yazılım Akışı

Takipçi robot `setup()` ile sensör/motor pinlerini kurar. `loop()` içerisinde HC-SR04 verisini okur. Mesafe eşiği aşılrısa motoru frenler ve otonom döngü için MQTT yayını yapar.

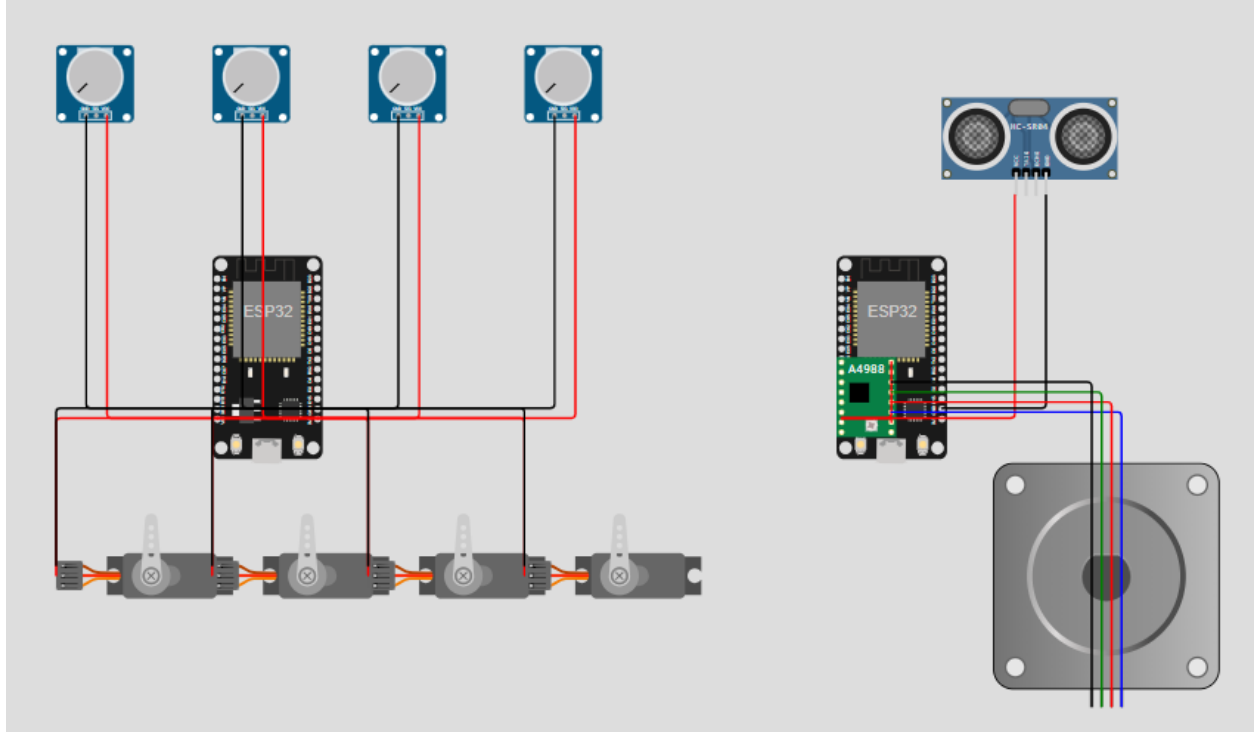


Şekil 3.3: Takipçi robot yazılım akışı

4. GERÇEKLEŞTİRİM

4.1 Donanım Montajı ve Devre Bağlantıları

Sistem ESP32 merkezli olup, motor güç hatları ile işlemci güç hatları optik izole şekilde tasarlanmıştır.



Şekil 4.1: Devre bağlantı şeması

4.1.1 Pin Bağlantı Tablosu

Bileşen / Pin	ESP32 GPIO	Bileşen Pini	Açıklama
HC-SR04 TRIG	14 (Takipçi)	TRIG	Sensör tetikleme sinyali.

HC-SR04 ECHO	18 (Takipçi)	ECHO	Yankı okuma sinyali.
L298N IN3	26 (Takipçi)	IN3	Motor ileri sürüş sinyali.
Gövde Servosu	25 (Lider)	PWM	Alt gövde eksen kontrolü.
Omuz Servosu	26 (Lider)	PWM	Ana kol uzanma kontrolü.
Potansiyometre	32-35 (Lider)	A0-A3	Manuel ADC referans okuma.

Tablo 4.1: ESP32 pin bağlantı tablosu

4.2 Yazılım Geliştirme Ortamı

IDE olarak Arduino 2.3.7, MQTT istemcisi olarak PubSubClient kütüphanesi kullanılmıştır. Broker olarak public HiveMQ, SCADA paneli olarak ise Node-RED kullanılmıştır.

4.3 Kurulum ve Yapılandırma Adımları

1. Kodlar derlenerek Lider ve Takipçi ESP32'lere yüklendi.
2. Cihazlar 2.4 GHz Wi-Fi ağına bağlandı.
3. Node-RED bilgisayarda ayağa kaldırılıp MQTT düğümleri HiveMQ'ya bağlandı.
4. UI Dashboard üzerinden göstergeler (Gauges) ve sürgüler (Sliders) eşleştirildi.

4.4 Robot Fotoğrafları



Şekil 4.2: Gerçekleştirilen robot prototipleri

5. TARTIŞMA

5.1 Bulgular ve Yorumlar

Sistem testlerinde Node-RED üzerinden gönderilen komutların 100-300 ms aralığında robot kola ulaştığı gözlenmiştir. Bu gecikme, ücretsiz bulut broker kullanımı için kabul edilebilir seviyededir. Lider ve Takipçi arasındaki M2M tetikleme senkronizasyonunda paket kaybı yaşanmamıştır.

5.2 Kısıtlamalar ve Karşılaşılan Sorunlar

En büyük sorun analog potansiyometre parazitlerinin ağa spam atarak DDOS etkisi yaratması olmuştur. Bu durum, yazılımsal 5 derecelik 'Ölü Bant' (Deadband) filtresi ile çözülmüştür.

6. SONUÇ VE ÖNERİLER

6.1 Sonuç

Çalışma sonucunda, M2M tabanlı, bulut kontrollü ve dağıtık çalışan başarılı bir siber-fiziksel otomasyon prototipi üretilmiştir. Donanım ve ağ kısıtlamaları yazılımsal IoT filtreleriyle aşılmıştır.

6.2 Gelecek Çalışmalar

1. Güvenlik için TLS/SSL sertifikasyonu eklenebilir.
2. Gecikmeyi sıfırlamak için yerel ağda Mosquitto Broker kurulabilir.

7. KAYNAKLAR

- [1] Espressif Systems, "ESP32 Technical Reference Manual," 2023.
- [2] J. Banks, "MQTT Essentials - A Lightweight IoT Protocol," Packt, 2020.
- [3] Node-RED, "Visual Programming for IoT," OpenJS Foundation, 2024.

EKLER

Ek A – Kaynak Kodu

```
#include <WiFi.h>

#include <PubSubClient.h>

#include <ESP32Servo.h>

const char* ssid = "Emirhan";

const char* password = "Emirhan54";

const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;

PubSubClient client(espClient);

Servo govdeServo;

Servo omuzServo;

Servo dirsekServo;

Servo kiskacServo;

const int govdePin = 25;

const int omuzPin = 26;

const int dirsekPin = 27;

const int kiskacPin = 16;

const int potGovde = 32;

const int potOmuz = 33;

const int potDirsek = 34;

const int potKiskac = 35;

// MERKEZ KONUM

int aciGovde = 57;

int aciOmuz = 180;

int aciDirsek = 111;

int aciKiskac = 1;

bool otomatikCalisiyor = false;
```

```

bool potModuAktif = true;

void logYaz(String mesaj) {

    client.publish("metanoia/sistem_log", mesaj.c_str());

}

void aciYayinla() {

    client.publish("metanoia/aci/govde", String(aciGovde).c_str());

    client.publish("metanoia/aci/omuz", String(aciOmuz).c_str());

    client.publish("metanoia/aci/dirsek", String(aciDirsek).c_str());

    client.publish("metanoia/aci/kiskac", String(aciKiskac).c_str());

}

void servoYaz(Servo &servo, int &aciDegiskeni, int yeniAci) {

    yeniAci = constrain(yeniAci, 0, 180);

    servo.write(yeniAci);

    aciDegiskeni = yeniAci;

    aciYayinla();

    delay(20);

}

void merkezeDon() {

    servoYaz(govdeServo, aciGovde, 57);

    servoYaz(omuzServo, aciOmuz, 180);

    servoYaz(dirsekServo, aciDirsek, 111);

    servoYaz(kiskacServo, aciKiskac, 1);

}

void setup_wifi() {

    WiFi.begin(ssid, password);

    Serial.print("WiFi baglaniyor");

    while (WiFi.status() != WL_CONNECTED) {

        delay(500);

        Serial.print(".");
    }
}

```

```
}

Serial.println();

Serial.println("WiFi baglandi");

Serial.print("IP: ");

Serial.println(WiFi.localIP());

}

void otomatikSekans() {

    otomatikCalisiyor = true;

    potModuAktif = false;

    logYaz("Robot kol otomatik isleme basladi.");

    // 1) Merkez

    logYaz("Robot kol merkez konuma geliyor.");

    merkezeDon();

    delay(1000);

    // 2) Egilme / nesneye yaklasma

    logYaz("Robot kol nesneye yaklasiyor.");

    servoYaz(govdeServo, aciGovde, 61);

    servoYaz(omuzServo, aciOmuz, 143);

    servoYaz(dirsekServo, aciDirsek, 109);

    servoYaz(kiskacServo, aciKiskac, 1);

    delay(1200);

    // 3) Govdeyi dondurerek nesneyi ittir

    logYaz("Robot kol nesneyi ittiriyor.");

    servoYaz(govdeServo, aciGovde, 106);

    servoYaz(omuzServo, aciOmuz, 133);

    servoYaz(dirsekServo, aciDirsek, 103);

    servoYaz(kiskacServo, aciKiskac, 1);

    delay(1500);

    // 4) Tekrar merkez
```

```

logYaz("Robot kol tekrar merkez konuma donuyor.");

merkezeDon();

delay(1000);

logYaz("Robot kol isini bitirdi. Bant calisabilir.");

client.publish("metanoia/durum", "BITTI");

otomatikCalisiyor = false;

potModuAktif = true;

}

void manuelPotKontrol() {

if (!potModuAktif || otomatikCalisiyor) return;

int yeniGovde = map(analogRead(potGovde), 0, 4095, 0, 180);

int yeniOmuz = map(analogRead(potOmuz), 0, 4095, 0, 180);

int yeniDirsek = map(analogRead(potDirsek), 0, 4095, 0, 180);

int yeniKiskac = map(analogRead(potKiskac), 0, 4095, 0, 180);

if (abs(yeniGovde - aciGovde) > 5) servoYaz(govdeServo, aciGovde, yeniGovde);

if (abs(yeniOmuz - aciOmuz) > 5) servoYaz(omuzServo, aciOmuz, yeniOmuz);

if (abs(yeniDirsek - aciDirsek) > 5) servoYaz(dirsekServo, aciDirsek, yeniDirsek);

if (abs(yeniKiskac - aciKiskac) > 5) servoYaz(kiskacServo, aciKiskac, yeniKiskac);

}

void callback(char* topic, byte* payload, unsigned int length) {

String mesaj = "";

for (int i = 0; i < length; i++) {

mesaj += (char)payload[i];

}

mesaj.trim();

String kanal = String(topic);

Serial.print("MQTT geldi | Topic: ");

Serial.print(kanal);

Serial.print(" | Mesaj: ");

```

```
Serial.println(mesaj);

if (kanal == "metanoia/komut" && mesaj == "AL") {

    otomatikSekans();

}

else if (kanal == "metanoia/mod") {

    if (mesaj == "POT") {

        potModuAktif = true;

        logYaz("Potansiyometre modu aktif.");

    }

else if (mesaj == "NODE") {

        potModuAktif = false;

        logYaz("Node-RED kontrol modu aktif.");

    }

}

else if (kanal == "metanoia/kontrol/govde") {

    potModuAktif = false;

    servoYaz(govdeServo, aciGovde, mesaj.toInt());

}

else if (kanal == "metanoia/kontrol/omuz") {

    potModuAktif = false;

    servoYaz(omuzServo, aciOmuz, mesaj.toInt());

}

else if (kanal == "metanoia/kontrol/dirsek") {

    potModuAktif = false;

    servoYaz(dirsekServo, aciDirsek, mesaj.toInt());

}

else if (kanal == "metanoia/kontrol/kiskac") {

    potModuAktif = false;

    servoYaz(kiskacServo, aciKiskac, mesaj.toInt());

}
```

```

    }
}

void reconnect() {

    while (!client.connected()) {

        Serial.print("MQTT baglaniyor... ");

        String clientId = "MetanoiaRobotKol-";

        clientId += String(random(1000, 9999));

        if (client.connect(clientId.c_str())) {

            Serial.println("baglandi");

            client.subscribe("metanoia/komut");

            client.subscribe("metanoia/mod");

            client.subscribe("metanoia/kontrol/govde");

            client.subscribe("metanoia/kontrol/omuz");

            client.subscribe("metanoia/kontrol/dirsek");

            client.subscribe("metanoia/kontrol/kiskac");

            logYaz("Robot kol ESP32 baglandi.");

            merkezeDon();

            aciYayinla();

        }

        else {

            Serial.print("hata: ");

            Serial.println(client.state());

            delay(3000);

        }

    }

}

void setup() {

    Serial.begin(115200);

    pinMode(potGovde, INPUT);

```

```
pinMode(potOmuz, INPUT);

pinMode(potDirsek, INPUT);

pinMode(potKiskac, INPUT);

ESP32PWM::allocateTimer(0);

ESP32PWM::allocateTimer(1);

ESP32PWM::allocateTimer(2);

ESP32PWM::allocateTimer(3);

govdeServo.setPeriodHertz(50);

omuzServo.setPeriodHertz(50);

dirsekServo.setPeriodHertz(50);

kiskacServo.setPeriodHertz(50);

govdeServo.attach(govdePin, 500, 2400);

omuzServo.attach(omuzPin, 500, 2400);

dirsekServo.attach(dirsekPin, 500, 2400);

kiskacServo.attach(kiskacPin, 500, 2400);

merkezeDon();

setup_wifi();

client.setServer(mqtt_server, 1883);

client.setCallback(callback);

}

void loop() {

    if (!client.connected()) {

        reconnect();

    }

    client.loop();

    manuelPotKontrol();

    delay(20);
```

2- Konveyör Bant Sistemi Kodu

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Emirhan";
const char* password = "Emirhan54";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);

const int trigPin = 5;
const int echoPin = 18;
const int in3Pin = 26;
const int in4Pin = 27;

const int ESIK_MESAFE = 15;

bool kolMesgul = false;
unsigned long sonAlgilamaZamani = 0;
const unsigned long beklemeSuresi = 5000;

void logYaz(const char* mesaj) {
  client.publish("metanoia/sistem_log", mesaj);
}

void bantCalistir() {
  digitalWrite(in3Pin, HIGH);
  digitalWrite(in4Pin, LOW);
}

void bantDurdur() {
  digitalWrite(in3Pin, LOW);
  digitalWrite(in4Pin, LOW);
}

int mesafeOku() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long sure = pulseIn(echoPin, HIGH, 30000);

  if (sure == 0) return 999;
```

```

int mesafe = sure * 0.034 / 2;
return mesafe;
}

void setup_wifi() {
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  String mesaj = "";
  for (int i = 0; i < length; i++) mesaj += (char)payload[i];
  mesaj.trim();

  if (String(topic) == "metanoia/durum" && mesaj == "BITTI") {
    kolMesgul = false;
    sonAlgilamaZamani = millis();

    bantCalistir();
    logYaz("Robot kol isini bitirdi. Bant calisiyor.");
  }
}

void reconnect() {
  while (!client.connected()) {
    String clientId = "MetanoiaLider-";
    clientId += String(random(1000, 9999));

    if (client.connect(clientId.c_str())) {
      client.subscribe("metanoia/durum");
      logYaz("Lider ESP32 baglandi. Bant calisiyor.");
    } else {
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);

  setup_wifi();

  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);
}

```

```
bantCalistir();
}

void loop() {
  if (!client.connected()) reconnect();

  client.loop();

  if (!kolMesgul && millis() - sonAlgilamaZamani > beklemeSuresi) {
    int mesafe = mesafeOku();

    Serial.print("Mesafe: ");
    Serial.println(mesafe);

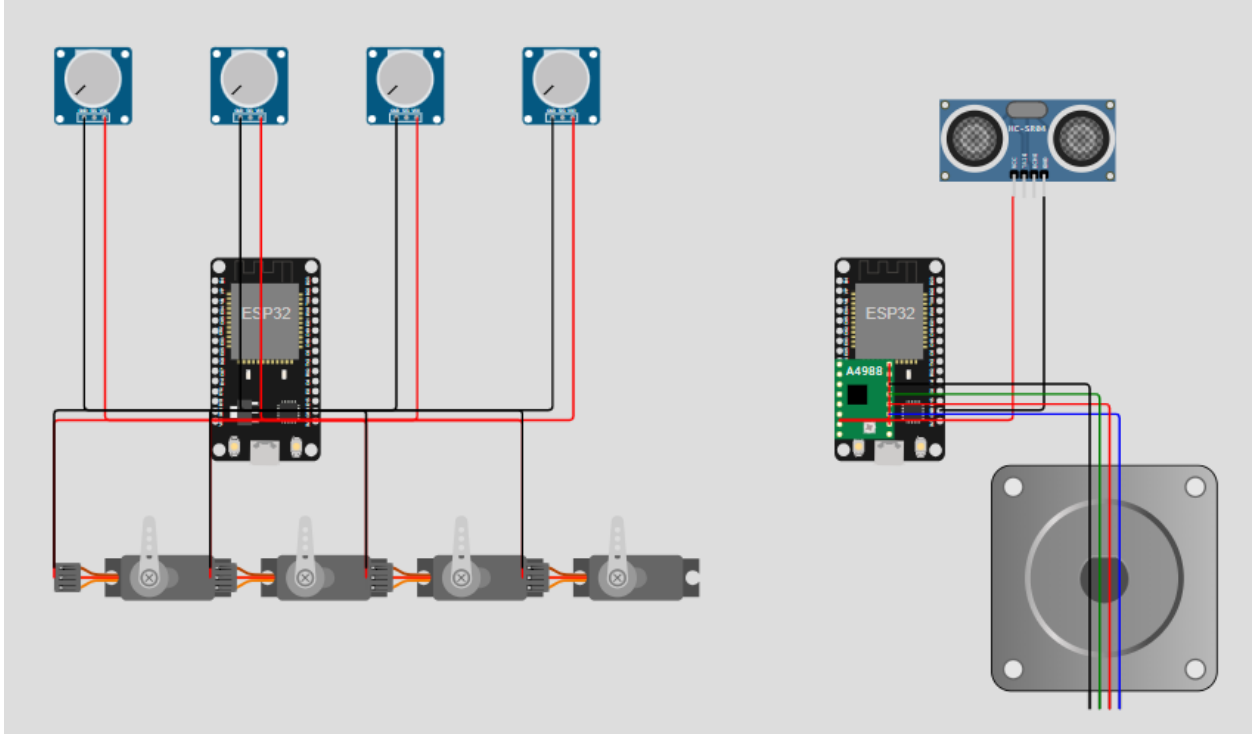
    if (mesafe > 0 && mesafe <= ESIK_MESAFE) {
      bantDurdur();

      kolMesgul = true;
      sonAlgilamaZamani = millis();

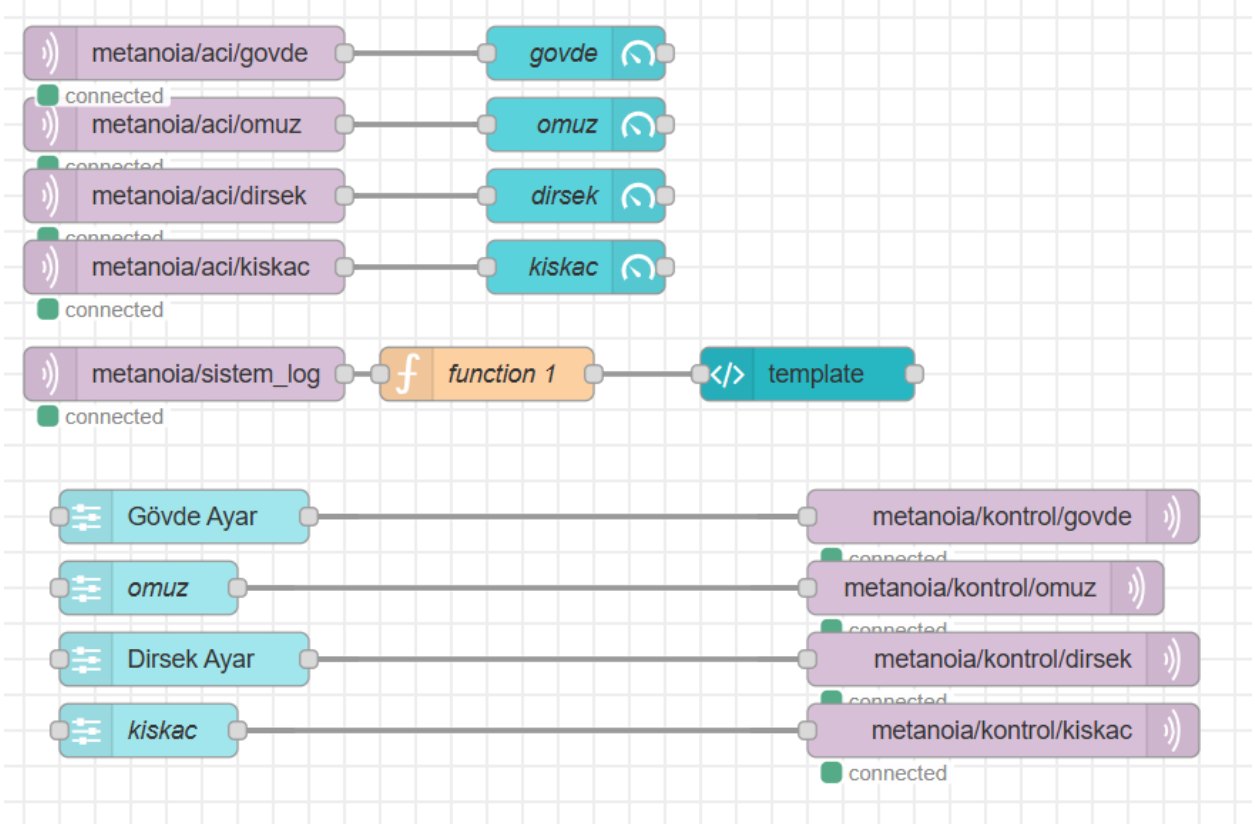
      client.publish("metanoia/komut", "AL");
      logYaz("Nesne algilandi, bant durdu. Kol cagirildi.");
    }
  }

  delay(100);
}
```

Ek B – Devre Şeması (Tam Boyut)



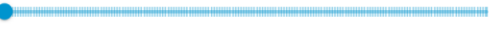
Ek C – Ham Test Verileri



Kontroller

Sistem bekleniyor... Yeni hareket algılanmadı.

Robot Kol Açıları

kiskac Ayar 

Dirsek Ayar 

omuz Ayar 

Gövde Ayar 

